

Bitacora de uso de IA.

Herramientas de IA utilizadas

Utilizamos Claude, de Anthropic, mediante su interfaz web claude.ai, como herramienta de apoyo para la organización del proyecto, generación de una base inicial de código, aclaración de dudas técnicas y redacción de documentación.

También utilizamos Gemini, de Google. Esta IA fue utilizada mayoritariamente en problemas de sintaxis.

No se utilizó ninguna otra herramienta de IA.

Prompts utilizados

Las peticiones centrales fueron:

- "Hola Claude, hoy serás un experto en ensambladores. Me gustaría que, siguiendo las siguientes instrucciones, generaras una base para construir un ensamblador de IA-32 en C, con analizador léxico y sintáctico manuales (sin lex/yacc), que soporte el subconjunto de instrucciones, registros y modos de direccionamiento del documento. *Nota: En este prompt también se anexó el documento proporcionado por la profesora*
- "Ayúdame a implementar dos versiones del ensamblador, una de 1 pasada y la otra de 2 pasadas." *Nota: Se proporcionó una base inicial de como estábamos tratando de desarrollar ambos ensambladores*
- "Implementa un mini enlazador que combine varios módulos objeto, resuelva extern/global y aplique relocalaciones ABS32/REL32."
- "Explica las decisiones de codificación: por qué cada opcode, cómo se arma ModRM y SIB."

Código generado automáticamente

La IA se utilizó para generar una base inicial del código del proyecto, que posteriormente fue revisada, corregida y validada por el equipo. Los archivos generados fueron:

Cabeceras en include/:

```
types.h, util.h, lexer.h, symtab.h, parser.h, encoder.h, objfile.h,  
asmcore.h.
```

Archivos fuente en src/:

`types.c`, `util.c`, `lexer.c`, `syntab.c`, `parser.c`, `encoder.c`, `objfile.c`, `asmcore.c`, `assembler1.c`, `assembler2.c`, `minilinker.c`.

Modificaciones realizadas manualmente

- Sofía: Revisó `lexer.c` y `parser.c`. Verificó la tokenización caracter por caracter, la distinción entre registros e identificadores y el parseo de operandos de memoria con base, índice, escala y desplazamiento. Confirmó que las etiquetas requieren `:` y que los comentarios con `;` se manejan correctamente. Escribió los casos de prueba `tests/test_mov_imm.asm`, `test_jumps.asm` y `test_forward.asm`.
- Ángel: Revisó `encoder.c`. Verificó manualmente la codificación de instrucciones contra la referencia Intel, por ejemplo `mov ebx, eax` → `8B D8` y `mov eax, [ebx+ecx*4+32]` → `8B 44 8B 20`. Escribió `tests/test_sib.asm` y `test_reloc.asm` para comprobar los modos de direccionamiento y las relocalaciones.
- Lair: Revisó `objfile.c` y `minilinker.c`. Se trazó el enlace de `mod1.o + mod2.o`, verificando las relocalaciones y comparando la salida con `build/program.hex`. Creó los archivos de ejemplo (`examples/mod1.asm`, `mod2.asm`, `test1.asm`), el `Makefile`, los scripts de compilación y pruebas (`scripts/`), la documentación del proyecto (`docs/`), el `README.md` y el `.gitignore`. También se encargó de notar detalles al momento de ejecución, mismos detalles que notificó al resto del equipo para su corrección.
- Lizeth: Revisó `syntab.c`, `asmcore.c`, `assembler1.c` y `assembler2.c`. Trazó el flujo de `assemble_stream` y verificó cómo `define_labels` controla la diferencia entre la versión de una pasada y la de dos pasadas. Probó la detección de redefinición de etiquetas y confirmó que se reporta error a `stderr` con código de salida 1. Escribió los casos de prueba `test_extern.asm` y `test_call.asm`.

Los módulos `util.c` y `types.c` se usaron prácticamente sin cambios, ya que solo contenían funciones auxiliares y definiciones generales.

Errores encontrados

Durante la compilación y las pruebas se encontraron los siguientes errores:

- Faltaban includes: `gcc` reportó advertencias de `implicit declaration of function 'free'` en `syntab.c`, `asmcore.c`, `assembler1.c` y `assembler2.c`. Se corrigió agregando `#include <stdlib.h>`.

- Indentación engañosa: En `encoder.c`, algunas líneas relacionadas con `ret` y no tenían `if/else` ni `return` en la misma línea, lo que generaba la advertencia `-Wmisleading-indentation`. Se corrigió separando las instrucciones en distintas líneas para hacer más claro el flujo.
- Colisión de nombres en una macro: En `minilinker.c`, la macro `RESOLVE` tenía un parámetro llamado `name`, que entraba en conflicto con el miembro `.name` de la estructura `Symbol`. Esto provocaba errores de compilación. Se corrigió reemplazando la macro por una función llamada `resolve_symbol`, usando el parámetro `want`.

Después de estas correcciones, el proyecto compiló sin advertencias con `-Wall -Wextra -std=c11`, y los 19 casos de prueba escritos por el equipo pasaron correctamente (`make test` → 19/19 OK).

- Errores Lógicos: En `parser.c`, la tokenización carácter por carácter no se ejecutaba como se esperaba, esto generaba que los archivos no se leyeran correctamente y nuestros test siempre resultaran fallidos. Se identificó que en la función `parse_mem`, no se estaba contando correctamente el número de tokens dentro de la memoria, por lo que se corrigió dicho error con condicionales.